

서울대학교 AI 빅데이터 CEO 과정 · 최종 발표

잠자던 데이터를 '컴파일된 지식'으로 깨우다

LLM Wiki 기반 건축 설계 지식 자산화 전략

RAG의 한계를 넘어,
AI가 직접 편집하는 사내 지식 체계로

(주)삼우종합건축사사무소
CAIO · 남효승

2026.05.07 · 최종 발표

한 장으로 보는 최종 제안

중간 발표의 비전은 유지, 핵심 엔진을 'RAG 챗봇'에서 'LLM Wiki'로 진화시킨다.



WHAT

LLM Wiki

원천 문서는 그대로 두고, AI가 별도 '컴파일된 지식 계층'을 직접 편집·관리한다. Confluence는 Source of Truth, Postgres가 LLM Wiki의 Canonical Store.



HOW

내부·통제·추적

외부 LLM API는 사용하지 않는다. 내부 GPU(L4×8 + L40S×8)와 사내 SSO 기반 RBAC, 그리고 Claim 단위 Citation으로 모든 답변을 원문까지 추적한다.



WHY

수익 가능

3년 TCO 약 5.4~7.2억(기존 GPU 활용 시), Base 시나리오 연 3.3억 원의 시간가치 창출. 2~3년 누적 기준 회수 가능, B2B 확장 시 Upside는 5억+.



같은 비전, 더 단단한 엔진 — 검증된 컨셉 위에서 '바로 시작 가능한' 단계로 들어선다

중간 발표 회고: 데이터 자산화에서 솔루션 비즈니스까지

‘잠자는 데이터 → AI 학습 재료 → 비즈니스 솔루션’이라는 3단계 비전은 그대로 유효하다.



중간 발표에서 제시한 핵심 수치 — 여전히 우리의 출발점

72.5

TB

축적 데이터

2,000

만 건

문서 건수

2.2

/ 5.0

데이터뱅크 만족도

1.8

억 / 년

유지보수 절감 잠재

그러나 ‘RAG 기반 기술 검색’만으로는 부족하다는 사실을, 우리는 지난 6개월간 직접 확인했다.

지난 6개월, 세 가지가 결정적으로 바뀌었다

기술적 한계, 규제 환경, 그리고 우리 자신의 학습 — 모두 LLM Wiki를 가리키고 있다.

체 감 성능 한 계

① RAG의 구조적 한계 확인

현장에서 직접 검증

ChatGPT MCP + Confluence CQL 병행 운용 결과:
매 질의마다 원문을 반복 검색하는 RAG 방식은
반복 질문에 비효율적이고, 답변 품질의 일관성이
낮으며,
복합 판단 (법규 + 공간 + 단계)에 약하다.

Compliance

② AI 기본법 시행 (2026.01.22)

거버넌스 요구 강화

고영향 AI 분류, 영향평가, 투명성·책임 의무.
발주처(공공기관·대기업) 대응을 위해서는
원천 추적성(claim → source)과 audit chain이
‘있으면 좋은 것’이 아니라 ‘없으면 안 되는 것’이 되
었다.

Paradigm Shift

③ Karpathy의 LLM Wiki 컨셉 등장

방향성 검증

AI가 원문을 읽고 ‘별도의 wiki 계층’을
지속적으로 편집·갱신한다는 새로운 패러다임.
원문은 immutable, wiki는 compiled —
바로 SAMOO의 Confluence + 설계 표준 환경에 정
확히 맞다.

RAG에서 LLM Wiki로: 같은 비전, 더 강한 엔진

중간 발표의 'RAG 기반 기술 검색'은 LLM Wiki라는 한 단계 진화한 구조로 흡수된다.

RAG 기반 검색 (중간 발표 시점)

동작 방식

질문 → 매번 원문 chunk 검색 → LLM이 즉석 답변 생성

한계

- 동일 질문에도 답변이 매번 달라진다 (재현성↓)
- 복합 판단 (법규+공간유형+설계단계)에 약하다
- 검색 chunk 단위로 답이 끊어진다
- 수만 건 문서에서 매번 검색은 비효율
- 출처 추적은 가능하나 'claim 단위'가 아니다



LLM Wiki (최종 제안)

동작 방식

AI가 원문을 미리 읽고 '컴파일된 지식 계층'을 별도 작성·갱신

강점

- 재현 가능 — 동일 질문에 일관된 답변 (revision 추적)
- Domain·Synthesis·Checklist 단위로 구조화된 답
- Claim 단위 citation — 문장 → 원문 위치까지 추적
- 사용 패턴 학습 → 새로운 '관점' 자동 컴파일 (주말 batch)
- AI Audit Chain — 어느 모델이 어느 prompt로 작성했나 추적

핵심: 원천 문서(Confluence)는 그대로 두고, AI가 '별도의 사내 지식 계층'을 직접 편집한다는 발상의 전환

건축 설계 지식 업무의 5대 요건에서 본 차이

검색 기술의 차이가 아니라, ‘설계 판단의 근거’를 다루는 방식의 차이다.

요건	일반 RAG / 검색	LLM Wiki (설계 지식 업무 요구)
업데이트	chunk / index 중심	개정일 · 버전 · 적용 범위까지 추적
답변	즉석 요약, 매번 새로 생성	반복 판단을 구조화해 축적 · revision 추적
근거	문서 단위 citation	Claim 단위 citation (page · table · cell)
권한	검색 후 필터 — 누출 위험	검색 결과 자체가 RBAC pre-filter 대상
첨부	동일 chunk로 일률 처리	PDF · Office · HWP · DWG별 parser 품질 관리

 결론: RAG는 보조 계층으로 유용하다. 단, 설계 기준·법규 지식의 본질은 LLM Wiki여야 한다.

“수백 문서에는 되지만 2,000만 문서에는?”

정확한 지적이다 — 그래서 SAMOO LLM Wiki는 Karpathy 원형을 그대로 쓰지 않는다.

KARPATHY 원형의 한계

개인 / 소규모용으로 설계됨

1 Context Window 의존

전체 wiki를 한 context에 로드하는 가정 — 50K~100K token 이상에서 붕괴

2 전수 컴파일 가정

모든 source를 wiki page로 만든다는 전제 — 2,000만 건에서는 실현 불가

3 단일 사용자 패턴

권한 분리 / 동시 편집 / 멀티 KB 미고려 — 850명 환경 부적합

SAMOO LLM WIKI의 회피 장치

처음부터 enterprise 규모로 설계

A 선별 컴파일 (Selective)

2,000만 source 모두 색인. 단, wiki page는 ~5K → ~50K로 제한 (0.025~0.25%)

B Hybrid 검색

wiki hit → 사전 컴파일 답변 / wiki miss → Layer 0 source retrieval로 fallback

C Adaptive 우선순위

어느 영역을 컴파일할지 사용 패턴이 결정 — query log + feedback signal 기반

D Session Synthesis

영구 wiki에 없는 질문도 retrieval + 즉석 합성 + topic_signal 기록 (다음 컴파일 후보)

 **핵심 차이:** Karpathy는 ‘전부 컴파일’을 가정한다. SAMOO는 ‘선별 컴파일 + Hybrid + Adaptive’ — 처음부터 다른 시스템이다.

LLM Wiki: AI가 직접 편집하는 사내 지식 백과사전

원문은 immutable Source, Wiki는 LLM이 유지하는 Compiled Knowledge — 둘은 분리된다.

Layer 0

Raw Source

Confluence 원문 · 첨부 (PDF/Office/HWP/DWG/Image)

Layer 1

Source Capsule

원문 1개 → 요약 + 핵심 claim + 적용 범위 + metadata

Layer 2

Domain Page

주제별 정리 — 피난, 방화, 주차, 장애인 편의, 병원, 주거, 오피스

Layer 3

Synthesis & Checklist

검토 절차 · 체크리스트 · 예외 · 충돌 기준 · 실무 판단 흐름

Bottom-up: 원문이 위로 갈수록 '업무 의사결정 친화적'으로 압축된다



Key Principle

AI는 직접 '쓴다.'

단, 쓰는 대상은 원천 Confluence가 아니라

AI 전용 Wiki 계층이다.

- 원천 Confluence는 read-only
- Wiki 계층은 revision · audit · citation · RBAC로 통제
- 사용자 질의·피드백으로 새 '관점'이 자동 생성

왜 LLM Wiki여야 하는가: 4가지 핵심 차별점

단순 '좋아 보이는 시스템'이 아니라, 한국 건축 도메인의 실무 요건과 정확히 맞물리는 구조다.



재현 가능한 답변

01

동일 질문에 동일 답변을 보장한다. revision · timestamp가 추적되므로 설계 검토 이력이 audit 가능. 시니어가 답변을 신뢰할 수 있다.



Claim 단위 Citation

02

문장 단위로 '어느 원문의 어느 위치(page·table·cell)에서 왔는지'가 기록된다. 법규·기준 검토에서 책임 추적이 가능한 구조.



관점별 컴파일

03

법규 중심, 공간 유형 중심, 설계 단계 중심... 같은 원문이라도 '바라보는 관점'에 따라 다른 wiki page가 만들어진다. 실무 친화.



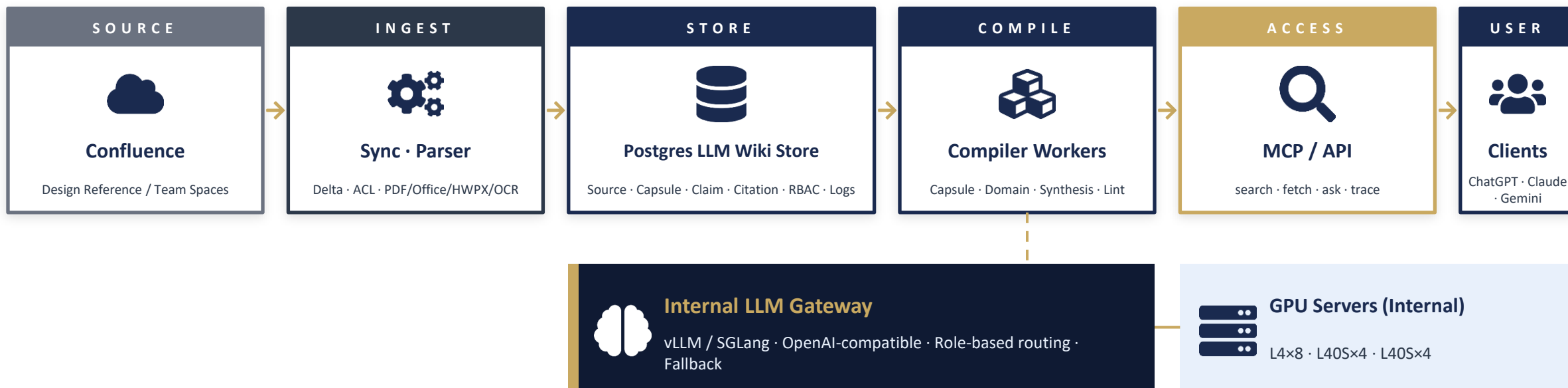
엔터프라이즈 거버넌스

04

외부 LLM API 0건. Samoo SSO + RBAC + AI 기본법 audit chain. 발주처(공공·대기업) 보안 요건을 처음부터 만족하는 설계.

목표 아키텍처: 5계층 구성, MVP는 4계층까지

Confluence(원천) → Postgres LLM Wiki Store(중심) → MCP/API → 다양한 LLM 클라이언트로 일원화.



원천

Confluence Cloud (immutable)

Canonical Store

Postgres + pgvector + pg_trgm

외부 LLM

사용 불가 — 내부 OSS 모델만

MVP 경계

Adaptive Compiler는 MVP 후

내부 GPU 운영: 역할별 라우팅과 Fallback 설계

외부 LLM API 제약을 전제로, ‘Interactive 경로는 절대 멈추지 않는다’는 원칙을 코드화한다.

GPU Pool	Primary Role	Fallback Role
L4 × 8 embedding · OCR · 보조	embedding, reranker, OCR/VLM 보조, 배치 전처리	vector score 생략, Postgres full-text search only
L40S × 4 #1 interactive 주력	interactive answer, source capsule, claim extraction	L40S #2로 자동 라우팅
L40S × 4 #2 compile · synthesis	weekly adaptive compiler, synthesis, planner agent	interactive 비상 시 #1 대신 응답 (off-peak compile 지연 허용)
Phase 0 모델 벤치마크 — 사내 질문 30~50개 기반 한국어 법규 이해 · 구조화 출력 · claim 추출 정확도 · citation 후보 선정 · long-context · latency · GPU memory · serving 안정성		

엔터프라이즈 거버넌스: 처음부터 '발주처가 신뢰할 수 있는 구조'

외부 LLM 배제 + Samoo SSO + RBAC + AI 기본법 audit chain — 4중 안전장치.



외부 LLM API 사용 불가

ChatGPT/Claude/Gemini API에 사내 데이터를 전송하지 않는다.
내부 GPU에 OSS 모델 (Qwen/Llama 계열) 직접 운영.
발주처 NDA·보안 요건과 충돌 없음.



Samoo SSO 통합 인증

전사 SSO(OIDC)로 인증, oid를 user primary key로 사용.
부서·팀·개인 단위 RBAC을 KB(Knowledge Base) 단위로 부여.
검색 결과조차 권한 필터링한다 (title/snippet/존재 여부 모두).



Derived ACL — 자동 권한 상속

Synthesis page는 인용된 source의 권한을 자동 상속한다.
전사 source A + 팀 전용 source B를 합치면, 전사용·팀용 page로
자동 분기 생성. 권한 오염 (data leak) 0건 목표.



AI 기본법 (2026.01 시행) 대응

claim → inference_run → model → prompt → source 5단계 audit chain.
'어느 모델이 어느 prompt로 어떤 원문을 보고 답했는가'를
언제든 재현·증명할 수 있는 구조.

Claim/Citation/Audit Chain — 한국 건축 도메인의 핵심 안전장치

‘이 답변이 어디서 왔는가’를 문장 단위로 답할 수 있어야, 법규·인허가 검토에 사용 가능하다.



실무 예시: 시니어가 답변을 검증하는 흐름

Q. “업무시설 지하주차장 피난계단 검토 시 무엇을 봐야 하나?”

A. Wiki Page «지하주차장 피난 검토 체크리스트» 반환

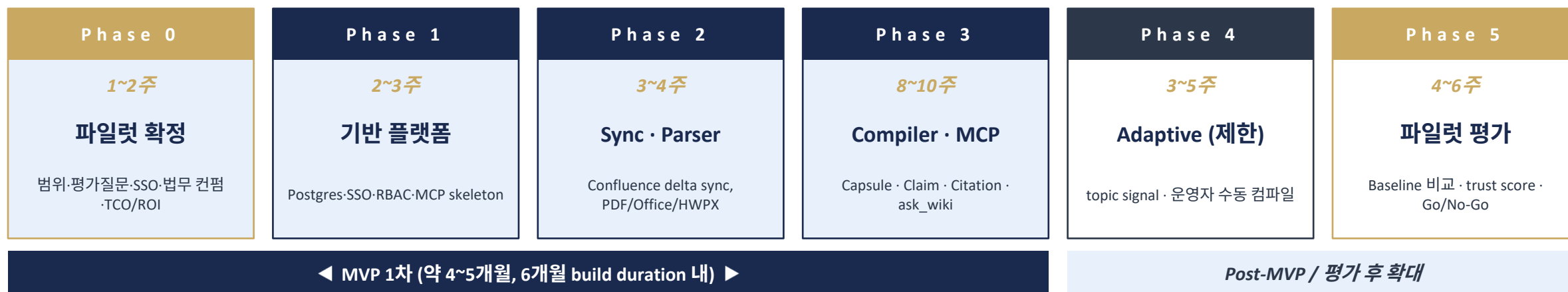
각 체크 항목은 다음과 같이 추적 가능:

- ▶ claim_id cl_847 → 「건축법 시행령 제48조」 PDF page 12, paragraph 3
- ▶ claim_id cl_312 → SAMOO 설계지침 v3.2 DOCX paragraph 84
- ▶ claim_id cl_592 → 서울시 건축 조례 PDF page 5, table 1, row 3

→ 시니어는 의심되는 항목만 ‘원문 클릭1화’로 즉시 확인

6단계 점진 추진 — MVP 우선, Adaptive는 평가 후 확대

Phase 0~3을 1차 MVP(약 4~5개월)로 묶고, 평가 결과로 Phase 4 자동화 확대 여부 결정.



🎯 Phase 5 Go/No-Go 성공 기준 — 평가 후 전사 확대 결정

전문가 답변 품질	Baseline 대비 품질 개선	Claim Citation 충족률	RBAC 권한 우회
평균 4.0 / 5.0 이상	20% 이상	90% 이상	0건 (필수)

기술 · 운영 · 현업 — 6대 리스크에 대한 대응 체계

기술 리스크에 한정하지 않고, 보안·운영·확산 리스크를 같은 비중으로 관리한다.

Risk	분류	Mitigation
내부 LLM 품질 부족	기술	사내 질문 30~50개 기반 모델 벤치마크 · 역할별 routing · L40S 양쪽 fallback
Parser 품질 편차	기술	parser status / parse_quality 저장 · metadata_only fallback · DWG/HWP는 단계적 확대
권한 오염 (data leak)	보안	Derived ACL · acl_propagation job · search-time RBAC · 권한별 page 분기
Citation 부정확	신뢰	Claim 단위 citation · lint rule · 전문가 blind 평가 · needs_review state
PII 로그 잔존	운영	retention sweep · PII redaction · 실패 시 CISO alert · 7일 지속 시 raw 저장 자동 비활성화
현업 확산 실패	확산	Power User Group 10~15명 · FAQ board · office hour · 분기 detractor 인터뷰

리스크의 60%는 '기술 리스크'가 아니다 — 우리가 통제할 수 있는 영역에서 먼저 시작한다.

AI 기본법 (2026.01 시행) 분류 및 재평가 Trigger

현 단계는 '직원 내부 도구'로 고영향 AI 비해당. 단, 외부 확장 시 자동 재평가하는 trigger 설정.

✓ 현 단계 분류

고영향 AI 비해당 (잠정)

- 직원 내부 참고/검색/지식관리 도구로 한정
- 자동 의사결정·외부 설계 자문 미제공
- 발주처 산출물에 직접 반영하지 않음
- **Phase 0에 사내 법무 컨펌 필수 (Go/No-Go Gate)**

그러나 우리는 이미 준비되어 있다

Audit Chain 5단계, claim 단위 citation, RBAC, retention policy

→ 고영향AI 분류 시에도 추가 작업이 최소화된다

재평가 Trigger (자동 발동)

- 1 발주처·외부 고객에게 직접 AI 답변 제공
- 2 구조·피난·안전·법규 적합성 자동 권고 도입
- 3 사람 검토 없는 자동 의사결정 workflow 편입
- 4 AI 답변이 인허가/법규 적합성 공식 산출물에 직접 반영
- 5 외부 서비스·프로젝트 납품물로 LLM Wiki 기능 제공

→ 어느 한 trigger라도 충족되면 분기 거버넌스 위원회에서 영향평가 즉시 착수

같은 목적지, 더 단단한 다리

중간 발표의 'B2B 솔루션 비즈니스' 비전은 그대로다. LLM Wiki는 그 비전에 도달하는 가장 빠르고 안전한 길이다.



2026

MVP 완성

전사 Design Reference

Phase 0~3 MVP 완성. 850명 Power User 확산.
파일럿 평가 4.0 / 5.0 달성.



2027

팀·개인 KB 확장

지식 자산 전사 표준화

팀별 / 프로젝트별 / 개인 KB 활성화.
Adaptive Compiler 운영 안정화 후 자동화 확대.



2028

B2B SaaS 출시

LLM Wiki SaaS Platform

타 설계사·CM사·발주처 대상 B2B 상품화.
'본업 R&D + IT 사업' 동시 영위 구조 완성.



‘설계 본업을 넘어선 IT 기술 기업으로의 포지셔닝 전환’ — 중간 발표 그대로의 약속

데이터 무덤에서, AI가 살아있는 지식 체계로

중간 발표의 비전을 LLM Wiki라는 단단한 엔진 위에 다시 세운다 — 이것이 본 발표의 전부다.

LLM Wiki가 SAMOO에게 주는 것

72.5	TB	잠자던 데이터의 깨어남
5.4 ~ 7.2	억	3년 TCO (보수적 추정)
331	M / 년	Base 시나리오 효과
0	건	외부 LLM API · RBAC 위반

CONCLUSION

비전은 그대로다.

데이터 자산화 → AI 활용 → 솔루션 비즈니스.

엔진은 진화한다.

RAG 챗봇이 아닌, 컴파일된 LLM Wiki로.

방향은 분명하다.

본업 R&D + IT 사업, 기술 컨트롤타워로의 도약.